Lecture 7: Data Manipulation and Analysis (I) – Tidyverse

Harris Coding Camp – Standard Track

Summer 2024

What is Tidyverse?

The tidyverse is a collection of R packages designed for data science with similar underlying philosophy and a common syntax.

Review: Installing and loading packages

Two steps to use a package:

- **Install it once** with the `install.packages()` function:

```
# Do this only once
install.packages("tidyverse")
```

- However, you need to **load a package every time** you plan to use it. To load a package, use the `library()` function:

```
# Add this to the top of your script / Rmd
library(tidyverse)
```

Tidyverse vs. Base R

Tidyverse	Base R	Operation
`select()`	`subset()` OR operators	extract columns
`filter()`	`subset()` OR operators	extract rows
`arrange()`	`sort()` OR `order()`	sort data
`rename()`	`names()`	rename the columns
`mutate()`	`$ + <-`	create new columns
`summarize()`	`summary()`, `aggregate()`	summarize data

Data manipulation with `dplyr`

The `tidyverse` package `dplyr` comprises many functions that perform mostly used data manipulation operations such as applying filter, selecting specific columns, sorting data, adding or deleting columns and aggregating data.

In this lecture, we will cover:

- `select()` to pick columns
- `arrange()` to sort the data
- `filter()` to pick rows that meet certain conditions

Selecting columns with `select()`

Selecting columns with `select()`

storms

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1999-06-13
Arthur	45	1010	1996-06-21

storm	pressure
Alberto	1007
Alex	1009
Allison	1005
Ana	1013
Arlene	1010
Arthur	1010

- In this example, we want to keep `storm` and `pressure`, and drop the other variables.

Selecting columns with `select()`

Use case: You want to present a subset of your **columns**.

Syntax: `select(data_name, var1, var2, var3, ...)`

```
select(housing_data, city, sales, listings)
#> # A tibble: 8,602 x 3
#>    city    sales listings
#>    <chr>   <dbl>    <dbl>
#>  1 Abilene    72      701
#>  2 Abilene    98      746
#>  3 Abilene   130      784
#>  4 Abilene    98      785
#>  5 Abilene   141      794
#>  6 Abilene   156      780
#>  7 Abilene   152      742
#>  8 Abilene   131      765
#>  9 Abilene   104      771
#> 10 Abilene   101      764
#> # i 8,592 more rows
```

```
# this works the same
select(housing_data, c(city, sales, listings))
```

Selecting columns with `select()`

The - says to exclude the columns listed in the vector.

```
select(housing_data, -city, -sales, -listings)
#> # A tibble: 8,602 x 6
#>     year month   volume median inventory  date
#>    <dbl> <dbl>    <dbl>  <dbl>     <dbl> <dbl>
#>  1  2000     1  5380000  71400       6.3 2000
#>  2  2000     2  6505000  58700       6.6 2000.
#>  3  2000     3  9285000  58100       6.8 2000.
#>  4  2000     4  9730000  68600       6.9 2000.
#>  5  2000     5 10590000  67300       6.8 2000.
#>  6  2000     6 13910000  66900       6.6 2000.
#>  7  2000     7 12635000  73500       6.2 2000.
#>  8  2000     8 10710000  75000       6.4 2001.
#>  9  2000     9  7615000  64500       6.5 2001.
#> 10  2000    10  7040000  59300       6.6 2001.
#> # i 8,592 more rows
```

```
# this works the same
select(housing_data, -c(city, sales, listings))
```

Selecting columns: Base R vs Tidyverse

Base R: `subset(data_name, condition, select)`

```
subset(housing_data, select = c(city, sales, listings))
```

- `data_name`: name of the data frame
- `condition` is the condition(s) indicating which **rows** to keep
- `select` indicates which **columns** to keep

Tidyverse: `select(data_name, var1, var2, ...)`

```
select(housing_data, city, sales, listings)
```

```
# this works the same
select(housing_data, c(city, sales, listings))
```

Sort rows with `arrange()`

Sort rows with `arrange()`

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1999-06-13
Arthur	45	1010	1996-06-21

storm	wind	pressure	date
Ana	40	1013	1997-07-01
Alex	45	1009	1998-07-30
Arthur	45	1010	1996-06-21
Arlene	50	1010	1999-06-13
Allison	65	1005	1995-06-04
Alberto	110	1007	2000-08-12

- Here, we want to sort the data by `wind` in ascending order.

Sort rows with arrange()

Order the data by year in **ascending** order (by default):

```
arrange(housing_data, year)
#> # A tibble: 8,602 x 9
#>    city      year month sales   volume median listings inventory  dat
#>    <chr>    <dbl> <dbl> <dbl>    <dbl>  <dbl>    <dbl>     <dbl> <dbl
#>  1 Abilene  2000     1    72  5380000  71400      701       6.3 2000
#>  2 Abilene  2000     2    98  6505000  58700      746       6.6 2000
#>  3 Abilene  2000     3   130  9285000  58100      784       6.8 2000
#>  4 Abilene  2000     4    98  9730000  68600      785       6.9 2000
#>  5 Abilene  2000     5   141 10590000  67300      794       6.8 2000
#>  6 Abilene  2000     6   156 13910000  66900      780       6.6 2000
#>  7 Abilene  2000     7   152 12635000  73500      742       6.2 2000
#>  8 Abilene  2000     8   131 10710000  75000      765       6.4 2001
#>  9 Abilene  2000     9   104  7615000  64500      771       6.5 2001
#> 10 Abilene  2000    10   101  7040000  59300      764       6.6 2001
#> # i 8,592 more rows
```

Sort rows with arrange()

To order the data in **descending** order, use `desc()`:

```
# Order the data by year in descending order
arrange(housing_data, desc(year))
#> # A tibble: 8,602 x 9
#>    city       year month sales   volume median listings inventory  da
#>    <chr>     <dbl> <dbl> <dbl>    <dbl>  <dbl>    <dbl>     <dbl> <db
#>  1 Abilene   2015     1   158 23486998 134100      801       4.4 201
#>  2 Abilene   2015     2   151 19834263 126500      767       4.1 201
#>  3 Abilene   2015     3   198 31869437 136800      821       4.4 201
#>  4 Abilene   2015     4   201 28301159 129600      891       4.7 201
#>  5 Abilene   2015     5   199 31385757 144700      919       4.8 201
#>  6 Abilene   2015     6   260 41396230 141500      965       5   201
#>  7 Abilene   2015     7   268 45845730 148700      986       5   201
#>  8 Amarillo  2015     1   204 33188726 138500     1120       4.3 201
#>  9 Amarillo  2015     2   188 34355428 149400     1084       4.2 201
#> 10 Amarillo  2015     3   317 53603130 140900     1051       3.9 201
#> # i 8,592 more rows
```

Sort rows: Base R vs Tidyverse

Base R: `data_name[order(data_name$var1, data_name$var2, ...),]`

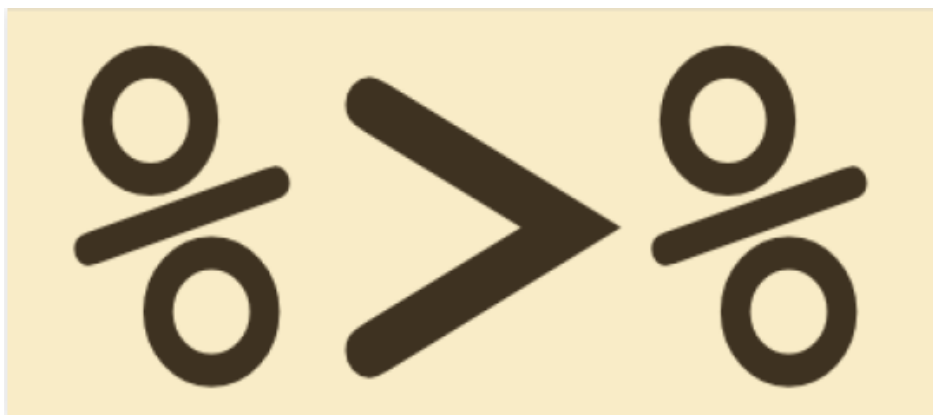
```
housing_data[order(housing_data$year), ]
```

Tidyverse: `arrange(data_name, var1, var2, ...)`

```
arrange(housing_data, year)

# Alternatively, we can use pipe operator
housing_data %>%
  arrange(year)
```

Introducing the pipe operator



Interlude: Ceci est une %>%

Pipes are a means of performing multiple steps in a single chunk of code.

Pipes are part of **Tidyverse** suite of packages, not Base R.

Whenever you see %>%, think of the words "and then..."

```
# Basic flow of using pipes
object %>%
  function_1 %>%
  function_2 %>%
  function_3
```

- Starting with `object`, execute `function_1`, and then execute `function_2`, and then execute `function_3`.

```
# The following works the same, but not very recommended
object %>% function_1 %>% function_2 %>% function_3
```

Interlude: Ceci est une %>%

Pipes work from left to right:

- The pipe operator %>% takes the left-hand side and makes it *input* in the right-hand side
- In other words, the object on the left of %>% is passed as the *first argument* to the function on the right of %>%

```
# Without pipes
select(housing_data, city, year, sales, volume)

# With pipes
housing_data %>%
  select(city, year, sales, volume)
```

Ceci est une %>%

We can chain together tidyverse functions to avoid making so many intermediate data frames:

```
housing_data %>%
  select(city, year, sales, volume) %>%
  arrange(desc(year))
#> # A tibble: 8,602 x 4
#>    city       year sales   volume
#>    <chr>     <dbl> <dbl>    <dbl>
#>  1 Abilene    2015   158 23486998
#>  2 Abilene    2015   151 19834263
#>  3 Abilene    2015   198 31869437
#>  4 Abilene    2015   201 28301159
#>  5 Abilene    2015   199 31385757
#>  6 Abilene    2015   260 41396230
#>  7 Abilene    2015   268 45845730
#>  8 Amarillo   2015   204 33188726
#>  9 Amarillo   2015   188 34355428
#> 10 Amarillo   2015   317 53603130
#> # i 8,592 more rows
```

We use `housing_data`, execute `select()`, and then execute `arrange()`.

Ceci est une %>%

We may want to insert **line breaks** to make long line of code more readable.

When inserting line breaks, pipe operator %>% should be the **last thing** before a line break, not the first thing after a line break:

```
# This works
housing_data %>%
  select(city, year, sales, volume) %>%
  arrange(desc(year))

# This does not work
housing_data
  %>% select(city, year, sales, volume)
  %>% arrange(desc(year))
```

Native pipe operator |>

R 4.1.0 introduced a **native pipe operator**, |>.

The behavior of the native pipe is by and large the same as that of the %>% pipe provided by tidyverse.

To learn more about the basic utility of base R pipes |>, see the pipe section of R for Data Science.

```
# This works
housing_data %>%
  select(city, year, sales, volume) %>%
  arrange(desc(year))

# This works the same
housing_data |>
  select(city, year, sales, volume) |>
  arrange(desc(year))
```

Try it yourself – Use tidyverse

Instead of using Base R, try to use tidyverse to redo the questions:

1. Create a new dataframe by extracting the columns `city`, `year`, `month`, `volume` and `median` from `housing_data`.
2. Sort the new dataframe by `median`.
3. Try doing parts 1 and 2 together using the **pipe operator**.

Choose rows with `filter()`

Choose rows that match a condition with `filter()`

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1999-06-13
Arthur	45	1010	1996-06-21

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Ana	40	1013	1997-07-01

- Here, we want to keep the data of storm `Alberto` and `Ana`, and drop the other rows.

Choose rows that match a condition with `filter()`

Syntax: `filter(data_name, conditions)`

Choose all the data from 2013 in Houston:

```
filter(housing_data, year == 2013 & city == "Houston")
```

Showing the first 6 rows:

```
#> # A tibble: 6 x 9
#>   city     year month sales     volume median listings inventory  da
#>   <chr>   <dbl> <dbl> <dbl>      <dbl>  <dbl>    <dbl>     <dbl> <db
#> 1 Houston  2013     1  4273  852045057 149500    21364       3.7 201
#> 2 Houston  2013     2  4886 1060985674 161900    21293       3.6 201
#> 3 Houston  2013     3  6382 1479273481 172300    20909       3.5 201
#> 4 Houston  2013     4  7116 1770746764 182400    20607       3.4 201
#> 5 Houston  2013     5  8439 2121508529 186100    20526       3.3 201
#> 6 Houston  2013     6  7935 2073909387 191600    21008       3.3 201
```

```
# this works the same
housing_data %>%
  filter(year == 2013 & city == "Houston")
```

Choose rows that match a condition with `filter()`

Choose all the data from 2013 in Houston and Austin:

```
housing_data %>%
  filter(year == 2013 & (city == "Houston" & city == "Austin"))
```

```
#> # A tibble: 0 x 9
#> # i 9 variables: city <chr>, year <dbl>, month <dbl>, sales <dbl>,
#> #   volume <dbl>, median <dbl>, listings <dbl>, inventory <dbl>, dat
```

- Nothing is returned here! Why?
- Because no row is recorded in Houston **AND** in Austin at any point!
- AND operator has different meaning from the conjunction 'and' in English

Choose rows that match a condition with `filter()`

Choose all the data from 2013 in Houston and Austin:

```
housing_data %>%
  filter(year == 2013 & (city == "Houston" | city == "Austin"))
```

```
#> # A tibble: 24 x 9
#>    city    year month sales    volume median listings inventory  dat
#>    <chr>  <dbl> <dbl> <dbl>     <dbl>  <dbl>    <dbl>     <dbl> <dbl
#>  1 Austin  2013     1  1566 402135669 199200     5548       2.6 2013
#>  2 Austin  2013     2  1795 468527290 206800     5752       2.6 2013
#>  3 Austin  2013     3  2384 661274904 217100     5848       2.6 2013
#>  4 Austin  2013     4  2692 820141712 227300     6121       2.7 2013
#>  5 Austin  2013     5  3182 944924177 228100     6424       2.8 2013
#>  6 Austin  2013     6  2970 880058970 234100     6724       2.9 2013
#>  7 Austin  2013     7  3376 993168216 227900     6721       2.8 2014
#>  8 Austin  2013     8  3318 931481472 218800     6706       2.7 2014
#>  9 Austin  2013     9  2544 725959392 224000     6569       2.6 2014
#> 10 Austin  2013    10  2350 648747783 218300     6114       2.4 2014
#> # i 14 more rows
```

```
# This works the same
housing_data %>%
  filter(year == 2013 & city %in% c("Houston", "Austin"))
```

Choose rows that match a condition: Base R vs Tidyverse

Base R: `subset(data_name, condition, select)`

```
subset(housing_data, year == 2013)
```

- `data_name` is dataset to be subset
- `condition` indicates which **rows** to keep
- `select` indicates which **columns** to keep

Tidyverse: `filter(data_name, condition)`

```
filter(housing_data, year == 2013)
# or use pipe operator
housing_data %>%
  filter(year == 2013)
```

Try it yourself – Use tidyverse

Instead of using Base R, try to use tidyverse to redo the questions:

1. Create a new dataframe from `housing_data` that includes observations with `listings` more than 5500 and `median` no less than 235000. Count the number of the selected observations.
2. Count the number of observations from Arlington, Bay Area and Collin County with `listings` less than 3000.
3. Filter all observations from Arlington, Bay Area and Collin County with `listings` at least 3500 in year 2013 and after, and only keep variables `city`, `year`, `month` and `listings` in your dataframe.

Recap

We learned:

- how to employ the 3 `dplyr` verbs of highest importance including
  - `select()` to pick columns
  - `filter()` to pick rows that meet certain conditions
  - `arrange()` to sort the data
- how to use %>% in `dplyr` contexts

Appendix: Review

Review: Relational operators in practice

```
4 < 4
#> [1] FALSE
```

```
4 >= 4
#> [1] TRUE
```

```
4 == 4
#> [1] TRUE
```

```
4 != 4
#> [1] FALSE
```

```
4 %in% c(1, 2, 3)
#> [1] FALSE
```

Review: Logical operators combine TRUEs and FALSEs

Operator	Description
!	not
&	and
|	or

```
# not true
! TRUE
#> [1] FALSE
```

```
# are both x & y TRUE?
TRUE & FALSE
#> [1] FALSE
```

```
# is either x | y TRUE?
TRUE | FALSE
#> [1] TRUE
```

Review: What do the following return?

Logical operators team up with relational operators.

- First, evaluate the relational operator
- Then, care out the logic.

```
# ! TRUE
! (4 > 3)

# TRUE & TRUE
(5 > 1) & (5 > 2)

# FALSE | FALSE
(4 > 10) | (20 < 3)
```

Review: What do the following return?

Logical operators team up with relational operators.

- First, evaluate the relational operator
- Then, care out the logic.

```
# ! TRUE
! (4 > 3)
#> [1] FALSE
```

```
# TRUE & TRUE
(5 > 1) & (5 > 2)
#> [1] TRUE
```

```
# FALSE | FALSE
(4 > 10) | (20 < 3)
#> [1] FALSE
```